Insertion en queue d’une liste chaînée

Les 4 versions : fonction / procédure, itérative / récursive

0. Rappels indispensables

Déclaration (version 2 du cours) : une **Liste** est un **pointeur** vers la première cellule, donc on écrit `l->donnee` et `l->suivant` (et jamais d’étoile après la flèche).

```
typedef struct cellule {
    TElement donnee;
    struct cellule *suivant;     /* adresse de la cellule suivante */
} Cellule;

typedef Cellule *Liste;          /* Liste = adresse d'une cellule */
```

Fonctions du prof : `donnee(l)` = `l->donnee`, `suivant(l)` = `l->suivant`, `estVide(l)` = `l == NULL`.

inserTete existe en deux versions, qu’on réutilise dans inserQueue :

```
/* version fonction : on renvoie la nouvelle tete */
Liste inserTete(TElement elt, Liste l) {
    Liste cel = (Liste) malloc(sizeof(Cellule));
    cel->donnee = elt;
    cel->suivant = l;        /* chainage A */
    l = cel;                 /* chainage B */
    return l;
}

/* version procedure : on modifie directement la variable de l'appelant */
void inserTete(TElement elt, Liste *l) {
    Liste cel = (Liste) malloc(sizeof(Cellule));
    cel->donnee = elt;
    cel->suivant = *l;       /* chainage A */
    *l = cel;                /* chainage B */
}
```

dernier (précondition : liste non vide) renvoie l’adresse de la dernière cellule :

```
Liste dernier(Liste l) {
    Liste der = l;
    while (!estVide(suivant(der)))
        der = suivant(der);
    return der;
}
```

L’idée clé : le compartiment `der->suivant` de la dernière cellule contient `NULL`, donc c’est une **liste vide**. Ajouter à la fin de la liste revient à ajouter dans cette liste vide, et on sait déjà le faire avec inserTete.

	Itérative	**Récursive**
Fonction (renvoie la Liste)	Version 1	Version 3
Procédure (Liste *l)	Version 2	Version 4

Version 1 : fonction, itérative

```
Liste inserQueue(TElement elt, Liste l) {
    if (estVide(l))
        return inserTete(elt, l);                 /* liste vide : insertion en tete */
    Liste der = dernier(l);                       /* adresse de la derniere cellule */
    der->suivant = inserTete(elt, der->suivant);  /* der->suivant vaut NULL */
    return l;                                     /* la tete n'a pas change */
}

/* Appel : */  l = inserQueue(8, l);
```

Comment ça marche. `inserTete(elt, der->suivant)` crée une cellule dont le *suivant* est `NULL` (la valeur de `der->suivant`) et **renvoie son adresse**. On range cette adresse dans `der->suivant` : c'est le chaînage.

Variante sans inserTete (même résultat) :

```
Liste inserQueue(TElement elt, Liste l) {
    if (estVide(l))
        return inserTete(elt, l);
    Liste der = dernier(l);
    Liste cel = (Liste) malloc(sizeof(Cellule));
    cel->donnee = elt;
    cel->suivant = NULL;     /* chainage A : ne jamais l'oublier */
    der->suivant = cel;      /* chainage B */
    return l;
}
```

Déroulé sur 3 -> 15, en insérant 8 :

Ligne	Effet
`estVide(l)`	non : la liste a 2 cellules
`der = dernier(l)`	der pointe vers la cellule « 15 »
`inserTete(8, der->suivant)`	crée « 8 » avec suivant = NULL, renvoie son adresse
`der->suivant = ...`	« 15 » pointe vers « 8 »
`return l`	la tête n'a pas changé : on renvoie l

Version 2 : procédure, itérative

```
void inserQueue(TElement elt, Liste *l) {
    if (estVide(*l))
        inserTete(elt, l);                  /* la tete elle-meme est vide */
    else {
        Liste der = dernier(*l);
        inserTete(elt, &(der->suivant));    /* "attention traduction en C" */
    }
}

/* Appel : */  inserQueue(8, &l);
```

Pourquoi `Liste *l` **?** Si la liste est vide, la tête change : il faut donc pouvoir modifier la variable de l'appelant, d'où l'adresse de la variable (`&l` à l'appel).

Pourquoi `&(der->suivant)` **et pas** `suivant(der)` **?** En algorithmique, le prof écrit `inserTete(elt, suivant(der))`. En C, `suivant(der)` renvoie une **copie** de l'adresse (la valeur NULL) : on modifierait la copie, pas la vraie cellule. Il faut donner à inserTete l'**adresse du compartiment**.

Ce qui se passe dans inserTete : le paramètre `l` reçoit `&(der->suivant)`, donc `*l` **est** `der->suivant` :

Ligne dans inserTete	Ce que ça fait vraiment
`cel->suivant = *l;`	`cel->suivant = der->suivant;` donc NULL
`*l = cel;`	`der->suivant = cel;` : **c'est le chaînage**

Variante sans inserTete :

```
void inserQueue(TElement elt, Liste *l) {
    if (estVide(*l))
        inserTete(elt, l);
    else {
        Liste der = dernier(*l);
        Liste cel = (Liste) malloc(sizeof(Cellule));
        cel->donnee = elt;
        cel->suivant = NULL;     /* chainage A */
        der->suivant = cel;      /* chainage B */
    }
}
```

Version 3 : fonction, récursive

```
Liste inserQueue(TElement elt, Liste l) {
    if (estVide(l))
        return inserTete(elt, l);
    l->suivant = inserQueue(elt, suivant(l));
    return l;
}

/* Appel : */  l = inserQueue(8, l);
```

Phrase clé : `inserQueue(elt, l)` **renvoie la liste l avec elt ajouté à la fin**.

Liste vide : ajouter à la fin d'une liste vide, c'est créer une liste d'une cellule (inserTete).

Liste non vide : la première cellule ne change pas, et on ajoute elt à la fin de la sous-liste `suivant(l)`.

Pourquoi `l->suivant = ...` **?** Une fonction ne modifie pas, elle **renvoie**. L'appel récursif renvoie la sous-liste avec elt en plus : il faut ranger ce résultat dans l'endroit d'où venait la sous-liste, c'est-à-dire `l->suivant`. Sans cette affectation, la nouvelle cellule serait créée mais reliée à rien. C'est la même logique que `l = inserTete(5, l);`.

Déroulé sur 3 -> 15, en insérant 8 :

Appel	l pointe vers	Action
1	cellule « 3 »	non vide : attend inserQueue(8, cellule15)
2	cellule « 15 »	non vide : attend inserQueue(8, NULL)
3	NULL	vide : crée « 8 » (suivant = NULL) et renvoie son adresse

Au retour :

Retour	Instruction exécutée
appel 2	`cellule15->suivant = adresse de « 8 »` : **le chaînage se fait ici**, puis renvoie la cellule « 15 »
appel 1	`cellule3->suivant = adresse de « 15 »` (déjà la valeur actuelle, sans effet), puis renvoie la cellule « 3 »

Seul le dernier appel change vraiment quelque chose ; les autres recopient la même valeur. On renvoie `l` à chaque niveau car la tête n'a pas changé.

Version 4 : procédure, récursive

```
void inserQueue(TElement elt, Liste *l) {
    if (estVide(*l))
        inserTete(elt, l);
    else
        inserQueue(elt, &((*l)->suivant));
}

/* Appel : */  inserQueue(8, &l);
```

Principe : on descend dans la chaîne en passant à chaque fois l'**adresse du compartiment** `suivant` de la cellule courante. Ce compartiment est traité comme une tête de liste. Quand on arrive sur un compartiment qui contient NULL, c'est une liste vide, et inserTete y range la nouvelle cellule.

Lecture de `&((*l)->suivant)` : `*l` est l'adresse de la cellule courante ; `(*l)->suivant` est son compartiment suivant ; & donne l'adresse de ce compartiment.

Déroulé sur 3 -> 15, en insérant 8 :

Appel	*l pointe vers	Action
1	cellule « 3 »	non vide : on descend sur &(cellule3->suivant)
2	cellule « 15 »	non vide : on descend sur &(cellule15->suivant)
3	NULL	vide : inserTete crée « 8 » et la range dans le compartiment cellule15->suivant

Aucun `return` n'est nécessaire : l'écriture se fait directement dans le bon compartiment.

Récapitulatif et pièges

Version	Type	Appel	Comment la tête revient
1	fonction, itérative	l = inserQueue(8, l);	return l
2	procédure, itérative	inserQueue(8, &l);	modification de *l
3	fonction, récursive	l = inserQueue(8, l);	return l (à chaque niveau)
4	procédure, récursive	inserQueue(8, &l);	modification de *l

Les pièges à éviter

• Oublier `cel->suivant = NULL;` dans les variantes sans inserTete : le compartiment contient n'importe quoi et le prochain parcours plante.

• Écrire `inserTete(elt, suivant(der))` en C : on modifie une copie. Il faut `&(der->suivant)`.

• Appeler `dernier(l)` sur une liste vide : sa précondition est « liste non vide », d'où le test `estVide` d'abord.

• Oublier le `&` à l'appel d'une version procédure, ou le `l =` à l'appel d'une version fonction.

• Écrire `sizeof(Liste)` au lieu de `sizeof(Cellule)` dans le malloc : Liste n'est qu'une adresse, il faut la place d'une cellule entière.

• Écrire `l.donnee` au lieu de `l->donnee` : en C, Liste est un pointeur (le « . » du cours en algorithmique devient « -> »).